Iterative Echo server and client

The purpose of this chapter is to provide an example that is both easy to understand (as a first example) and meets "Production Grade" standards in terms of efficiency
This example describes an

There are several models [= "ways"] to implement a TCP Echo client server application.
Or: several models to implement the server side of an Echo client server application.

In this example we describe a synchronous TCP Echo server that utilizes proper buffer management and comprehensive error handling.

1. Server Code (server.c)

The server binds to a local port and waits for incoming connections.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <errno.h>

#define PORT 8080
#define BUFFER_SIZE 4096 // Standard Linux memory page size for maximum efficiency


int main() {
    int server_fd, client_fd;
    struct sockaddr_in address;
    int opt = 1;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE];
```

```
    // 1. Create the Socket (File Descriptor)
    // AF_INET = IPv4, SOCK_STREAM = TCP
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }


    // 2. Set Socket Options - Efficiency and Reusability
    // SO_REUSEADDR allows the server to restart immediately without waiting
    // for the kernel's TIME_WAIT state to expire.
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT, &opt,
sizeof(opt))) {
        perror("setsockopt");
        exit(EXIT_FAILURE);
    }

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY; // Listen on all network interfaces
(including localhost)
    address.sin_port = htons(PORT);       // Convert to Network Byte Order (Big Endian)


    // 3. Bind - Linking the socket to an address and port
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
    }


    // 4. Listen - Putting the socket in passive mode to wait for connections
    // Backlog is set to 3 (the number of pending connections allowed in the queue)
    if (listen(server_fd, 3) < 0) {
        perror("listen");
        exit(EXIT_FAILURE);
    }

    printf("Server listening on port %d...\n", PORT);
```

```
    while(1) {
        // 5. Accept - Block until a new client arrives.
        // A new FD is created specifically for this conversation.
        if ((client_fd = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen)) <
0) {
            perror("accept");
            continue; // Continue trying to accept other clients
        }

        printf("New client connected\n");

        // 6. Echo Loop - Read and Write
        // Efficiency: Using a single buffer and avoiding unnecessary copies.
        ssize_t bytes_read;
        while ((bytes_read = read(client_fd, buffer, BUFFER_SIZE)) > 0) {
            // Send the data back exactly as it was received
            send(client_fd, buffer, bytes_read, 0);
        }

        if (bytes_read == 0) {
            printf("Client disconnected\n");
        } else if (bytes_read < 0) {
            perror("read error");
        }

        close(client_fd); // Close the current client's FD
    }

    close(server_fd);
    return 0;
}
```

2. Client Code (client.c)

The client sends a message and prints the response received from the server.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define BUFFER_SIZE 4096

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char *hello = "Hello from client! This message should be echoed.";
    char buffer[BUFFER_SIZE] = {0};


    // 1. Create the Socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);


    // 2. Convert IP address from text to binary structure
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        printf("\nInvalid address/ Address not supported \n");
        return -1;
    }


    // 3. Connect to the server
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }
```

```
    // 4. Send Data
    send(sock, hello, strlen(hello), 0);
    printf("Message sent to server\n");

    // 5. Receive Echo from server
    int valread = read(sock, buffer, BUFFER_SIZE);
    if (valread > 0) {
        printf("Echo received: %s\n", buffer);
    }


    // 6. Close the connection
    close(sock);
    return 0;
}
```

Why is this code efficient?

1. **Buffer Size Alignment:** We defined BUFFER_SIZE as 4096 bytes. This is the standard Page size in most Linux architectures. Performing read and write operations in multiples of the page size maximizes I/O efficiency with the Kernel.
2. **Zero-Copy Logic (Conceptual):** In the Echo code, we do not process the string. Data enters the buffer and is sent out immediately. In advanced Linux versions, one could use splice() to perform a direct transfer within the Kernel without copying to User Space; however, for a first example, using a fixed buffer is the correct balance.
3. **SO_REUSEADDR:** A critical feature in Linux systems. Without this flag, if the server crashes or closes, the port will remain "stuck" in the TIME_WAIT state for several minutes, preventing an immediate restart.

Notes:

- **The difference between socket and accept.**
- **Byte Order:** The htons (Host to Network Short) function. Big-Endian vs. Little-Endian in networking.
- **Blocking vs. Non-blocking:** In this example, read and accept are blocking calls.